Distributed Systems

11th Lecture

Eom, Hyeonsang (엄현상)
Department of Computer Science & Engineering
Seoul National University

©Copyrights 2026 Eom, Hyeonsang All Rights Reserved

Outline

- Other Topics in Distributed Systems
  - Consistency
  - Replication
  - Fault Tolerance
- Q&A

Summary of Data-Centric Consistency Models

Models Not Using Synchronization Operations

Consistency	Description
Strict	Absolute time ordering of all shared accesses matters.
Linearizability	All processes must see all shared accesses in the same order. Accesses are furthermore ordered according to a (nonunique) global timestamp
Sequential	All processes see all shared accesses in the same order. Accesses are not ordered in time
Causal	All processes see causally-related shared accesses in the same order.
FIFO	All processes see writes from each other in the order they were used. Writes from different processes may not always be seen in that order

Summary of Data-Centric Consistency Models

Models Using Synchronization Operations

Consistency	Description
Weak	Shared data can be counted on to be consistent only after a synchronization is done
Release	Shared data are made consistent when a critical region is exited
Entry	Shared data pertaining to a critical region are made consistent when a critical region is entered.

Client-Centric Consistency Models

- Eventual Consistency
- Monotonic Reads
- Monotonic Writes
- Read Your Writes
- Writes Follow Reads
- Implementation

Eventual Consistency

- All Replicas Will Gradually Become Consistent

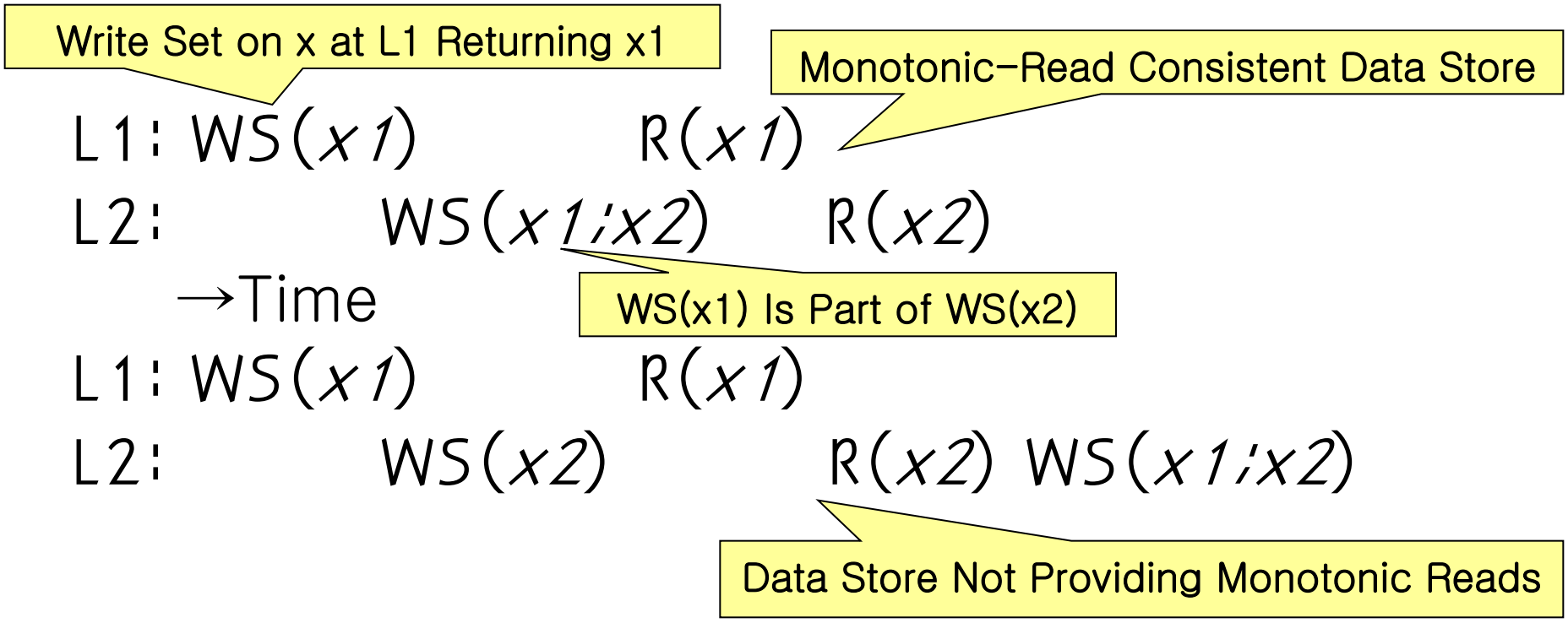
Distributed Data Stores Characterized by the Lack of Simultaneous Updates: e.g., DNS and WWW

Figure 7-11 on Page 290

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

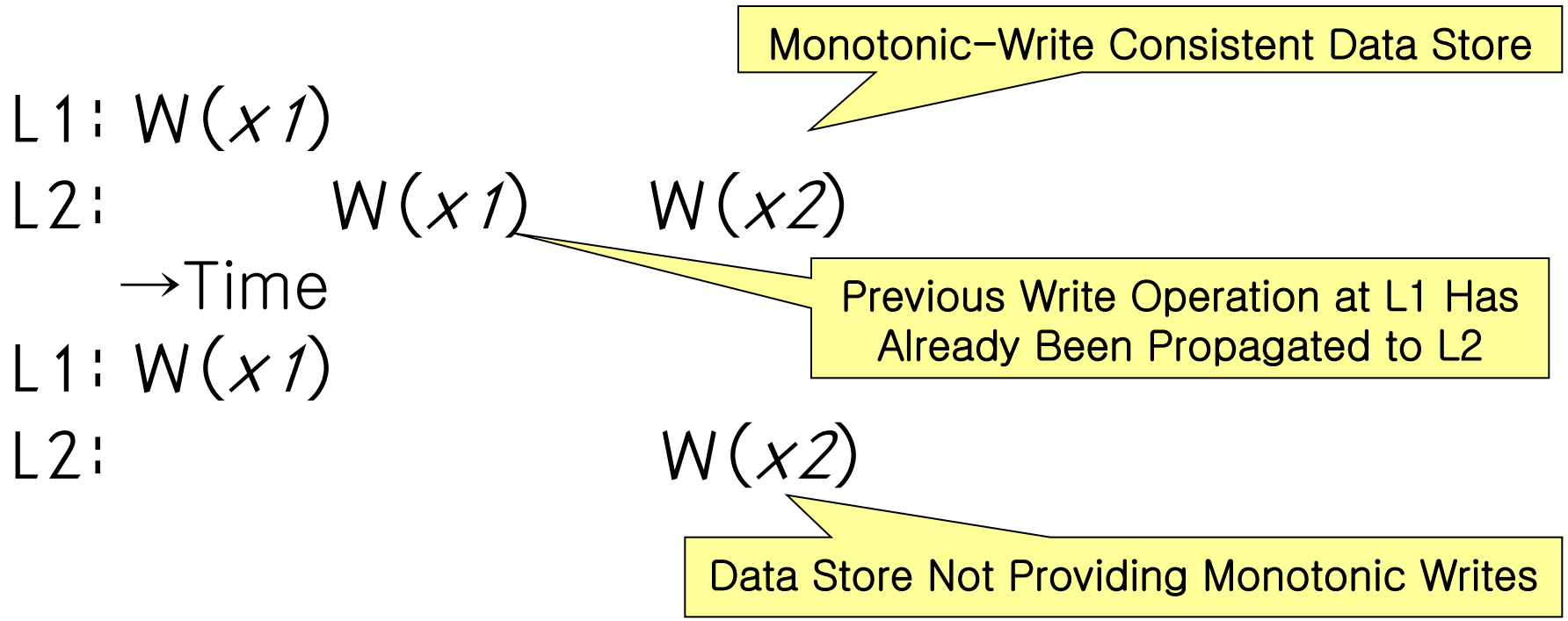
Monotonic-Read Consistency

- If a Process Reads the Value of a Data Item *x*, Any Successive Read Operation on *x* by That Process Will Always Return That Same Value or a Recent Value



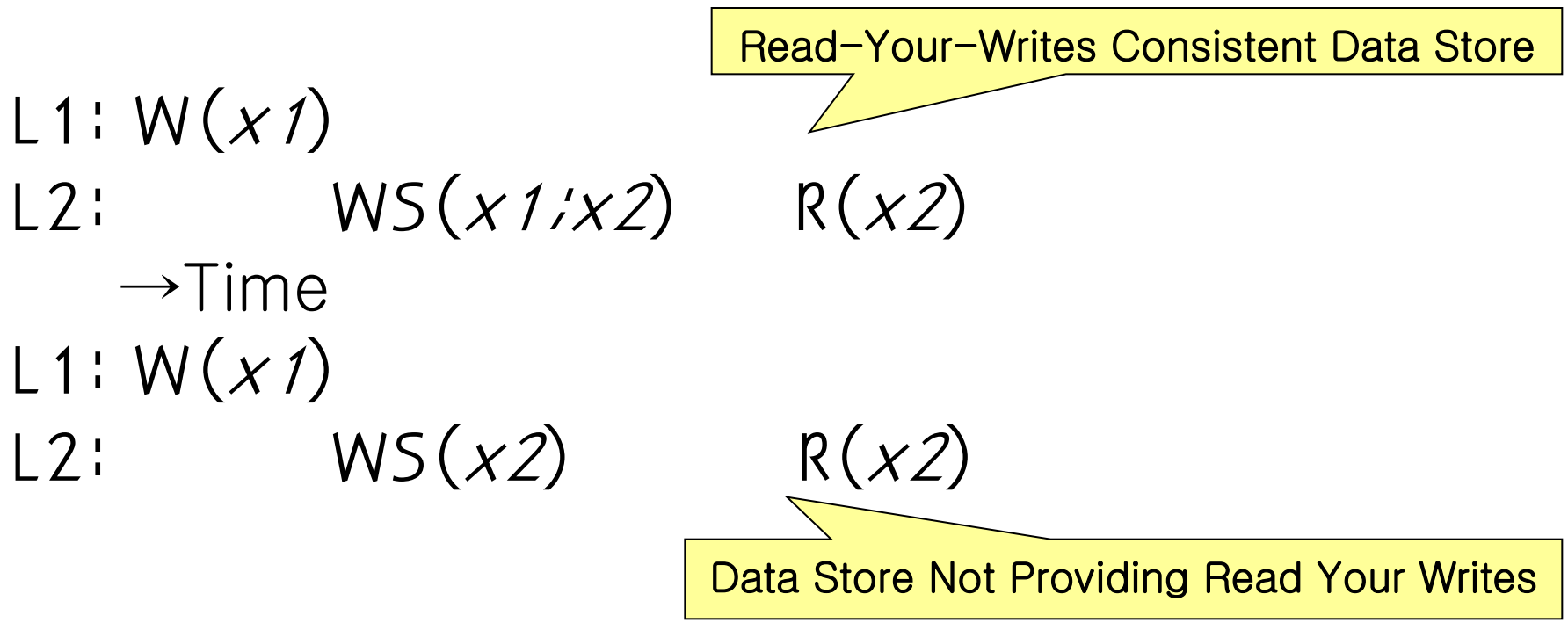
Monotonic-Write Consistency

- A Write Operation by a Process on a Data Item x, Is Completed before Any Successive Write Operation on x by the Same Process



Read-Your-Writes Consistency

- The Effect of a Write Operation by a Process on Data Item *x* Will Always Be Seen by a Successive Read Operation on *x* by the Same Process



Writes-Follow-Reads Consistency

- A Write Operation by a Process on Data Item *x* Following a Previous Read Operation on *x* by the Same Process Is Guaranteed to Take Place on the Same or a More Recent Value of *x* That Was Read

Writes-Follow-Reads Consistent Data Store

L1: WS(x1) R(x1)

L2: WS(x1;x2) W(x2)

→Time

L1: WS(x1) R(x1)

L2: WS(x2) W(x2)

Data Store Not Providing Writes Follow Reads

Implementation

- Assumptions
  - Each write operation is assigned a globally unique identifier by the server that accepts the operation for the first time (i.e., the operation is initiated at that server)
  - Read set for a client consists of the write identifiers relevant for the read operations performed by a client
  - Write set consists of the writes performed by the client

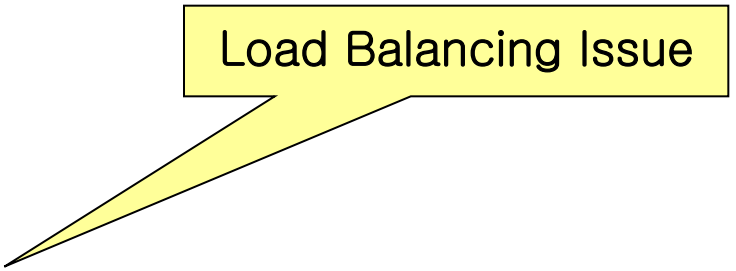
Implementation (Cont'd)

- Monotonic-Read Consistency
  - When a client performs a read at a server, that server is handed the client's read set to check whether all the identified writes have been taken place locally
  - If not, the read is postponed
    - Alternatively, the read is forwarded to a server where the writes have already taken place
- Monotonic-Write Consistency
  - When a client performs a write at a server, that server is handed the client's write set to check whether all the identified writes have been performed first and in correct order

Implementation (Cont'd)

- Read-Your-Writes Consistency
  - When a client performs a read at a server, that server is handed the client's write set to check whether all the identified writes have been taken place locally
- Writes-Follow-Reads Consistency
  - When a client performs a write at a server, that server is handed the client's read set to check whether all the identified writes have been taken place locally

Replica Placement



- Permanent Replicas
  - Distribution of a Web site
    - Replication across servers
    - Mirroring
- Server-Initiated Replicas
- Client-Initiated Replicas
  - (Client) caches

Server-Initiated Replicas

File Migration

Figure 7-18 on Page 300

Counting access requests from different clients.

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Pull vs Push Protocols

Issue	Push-based	Pull-based
State of server	List of client replicas and caches	None
Messages sent	Update (and possibly fetch update later)	Poll and update
Response time at client	Immediate (or fetch-update time)	Fetch-update time

A comparison between push-based and pull-based protocols in the case of multiple client, single server systems.

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Consistency Protocols

- Protocols for Sequential Consistency Classified by Whether or Not There Is a Primary Copy of data to Which All Write Operations Should Be Forwarded
  - Primary-Based Protocols
  - Replicated-Write Protocols
    - A write operation can be initiated at any replica

Remote-Write Protocols

Figure 6-27 on Page 338

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, Int. e, (c) 2002 Prentice-Hall, Inc. All rights reserved

Remote-Write Protocols (Cont'd)

Figure 7-20 on Page 309

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Local Write Protocols

Figure 6-29 on Page 340

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, Int e. (c) 2002 Prentice-Hall, Inc. All rights reserved

Local Write Protocols (Cont'd)

Figure 7-21 on Page 310

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Active Replication

- An Operation Is Forwarded to Any Replica with an Associated Process Carrying Out Update Operations
  - Potential Problems
    - Operations need to be carried out in the same order everywhere
      - Totally ordered multicast mechanism
      - Sequencer, a central coordinator
    - Invocations can be replicated

Scalability Problem!

Replicated Invocation Problem

Figure 6-31 on Page 342

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, Int. e, (c) 2002 Prentice-Hall, Inc. All rights reserved

Replicated Invocation Solution

Figure 6-32 on Page 343

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, Int. e, (c) 2002 Prentice-Hall, Inc. All rights reserved

Quorum-Based Protocol

- Idea: To Require Clients to Request and Acquire the Permission of Multiple Servers before Carrying Our Any Operation
  - To update a file, a client must first contact at least half the server plus one (a majority) and get them to agree to do the update
  - To read a replicated file, a client must also contact at least half the servers plus one and ask them to send the version numbers associated with the file

Fault Tolerance

- Dependability
  - Availability
    - Ready to be used immediately
  - Reliability
    - Running continuously without failure
  - Safety
    - Not leading to catastrophe in the case of temporary failure
  - Maintainability
    - How easy a failed system can be repaired

Failure Modes

Figure 8-1 on Page 324

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Failure Masking by Redundancy

Figure 8-2 on Page 327

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Process Resilience

- Flat Groups vs Hierarchical Groups

Advantage: No Single Point of Failure
Disadvantage: Complicated Decision Making

Communication in a flat group

Figure 8-3 on Page 329

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Agreement in Faulty Systems

- Two-Army Problem: Agreement Even between TWO Processes Is Not Possible in the Face of Unreliable Communication
- Byzantine Generals Problem
  - Whether to reach an agreement with loyal generals and traitors

Agreement in Faulty Systems (Cont'd)

Figure 8-5 on Page 333

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Agreement in Faulty Systems (Cont'd)

Figure 8-6 on Page 334

A System with m Faulty Processes, Agreement Can Be Achieved Only If 2m+1 Correctly Functioning Processes Are Present, for a Total of 3m+1

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved